🔬 Phase 2: Data Mining & Reverse Engineering

Goal: Extract the exact mathematical constants (Floating-point values) used by the simulation engine by analyzing 30,000 historical race records.

📌 Phase Overview

The core challenge of this project is to discover the hidden formula used to calculate lap times. Since the simulation is 100% deterministic, we must isolate variables from the historical JSON data to find the precise values for Tire Offsets, Degradation Rates, and Temperature Multipliers.

🛠️ Task 2.1: Build the Data Extraction Script

Objective: Create a Python utility to parse and process the large historical dataset.

- **Location:** `scripts/analyzer.py`
- **Methodology:** * Iterate through `data/historical_races/races_00000-01999.json`.
  - Extract `base_lap_time`, `track_temp`, and the `finishing_positions`.
  - Since the simulation is independent (virtual lanes), we can deduce that a driver's total time is the sum of their individual lap times.
- **Deliverable:** A script capable of grouping race data by track and temperature for comparison.

🧪 Task 2.2: Isolate Compound Offsets (O_c)

Objective: Determine the speed difference between SOFT, MEDIUM, and HARD tires.

- **Theory:** Each tire compound has a "base speed" offset relative to the track's `base_lap_time`.
- **Method:** * Find a race where different drivers started on different compounds (e.g., Driver A on SOFT, Driver B on MEDIUM).
  - Look at **Lap 1** specifically (where Tire Age is exactly 1).
  - **Formula:** $Lap1_Time - Base_Lap_Time - (1 \times Degradation)$.
  - *Note:* By comparing two drivers in the same race on Lap 1, the degradation effect is identical, allowing us to see the pure "Offset" gap between SOFT, MEDIUM, and HARD.
- **Target Output:** * `SOFT_OFFSET`: (e.g., -0.500s)
  - `MEDIUM_OFFSET`: (e.g., 0.000s)
  - `HARD_OFFSET`: (e.g., +0.800s)

📉 Task 2.3: Isolate Base Degradation Rate (D)

Objective: Calculate how many seconds a tire loses per lap of age.

- **Theory:** As `tire_age` increases, the lap time increases linearly.
- **Method:** * Analyze a single driver's "stint" (the laps between pit stops).
  - Measure the difference between consecutive laps.

    - **Formula:** $Degradation = Time_{Lap(n+1)} - Time_{Lap(n)}$.
    - By averaging this delta across thousands of laps, we find the constant D.
- **Target Output:** The exact floating-point value added to the time for every increment in tire age.

🌡️ Task 2.4: Isolate Temperature Coefficient (S_t)

Objective: Determine how track temperature scales the tire wear.

- **Theory:** Higher track temperatures accelerate tire degradation.
- **Method:** * Find two races on the **same track** with the **same** `base_lap_time` but different `track_temp` (e.g., 20°C vs 35°C).
    - Compare the Degradation Rate (D) calculated in Task 2.3 for both races.
    - Observe the ratio of increase in wear per degree of Celsius.
- **Target Output:** A multiplier formula, likely in the form of: $Wear = Base_Wear \times (1 + (Temp \times Coefficient))$.

🏁 Final Deliverable: The Master Formula

At the end of Phase 2, you should be able to complete this function:

```
def calculate_lap_time(base_time, compound, age, temp):
    offset = OFFSETS[compound]
    wear = (age * BASE_WEAR) * (temp * TEMP_FACTOR)
    return base_time + offset + wear
```